

Name: P. Karpaga Shree

Subject name: compiler design

Register number: 192424040

Date: 01/08/2026

Subject code: 25AD407

① Given

$E \rightarrow E + E \mid id$

$E \rightarrow E + E, E \rightarrow +EE'$

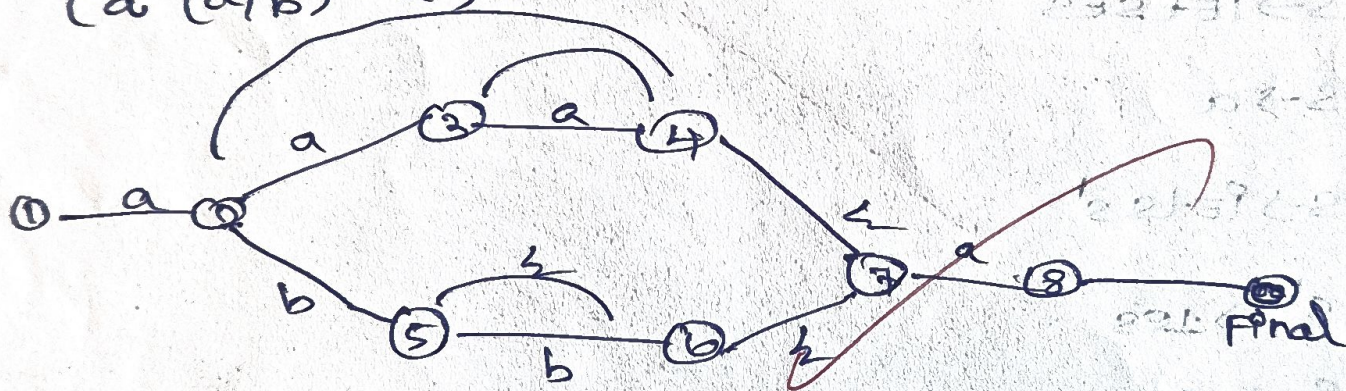
$E \rightarrow id$

$E' \rightarrow +E', E' \rightarrow +EE'$

78
100

② NFA for regular expression

$(a(a/b)^*a)a$

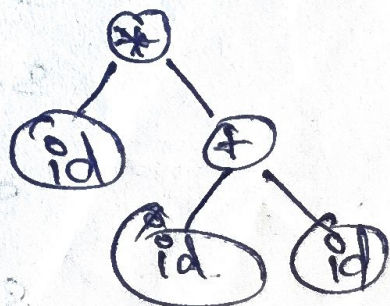


Accepting state:

aa
aaa
aba
aaaa
aaba
aaaaa
abba

Rejected state:

a
b
ba
ab



③ $id + id * id$

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$
 $T' \rightarrow *FT \mid \epsilon$
 $F \rightarrow (E) id$

④ Eliminate left recursion

$$A \rightarrow A\alpha / B$$

$$A \rightarrow A\alpha$$

$$A \rightarrow B$$

$$A \rightarrow BA'$$

$$A' \rightarrow \alpha A' / \epsilon$$

⑤ Predictive parsing

ii) Apply left factoring

$$S \rightarrow \text{PETS} / \text{PETSes} / a$$

$$S \rightarrow \text{PETS}$$

$$S \rightarrow \text{PETSes}$$

$$S \rightarrow a$$

$$S \rightarrow \text{PETS} s'$$

$$s' \rightarrow es$$

$$S \rightarrow a$$

i) predictive parsing

$$S \rightarrow \text{PETS} / \text{PETSes} / a$$

$$S \rightarrow \text{PETS}$$

$$S \rightarrow \text{PETSes}$$

$$S \rightarrow a$$

$$S \rightarrow \text{PETS} s'$$

$$s' \rightarrow es$$

$$S \rightarrow a$$

⑥ calculate the first and follow.

$$S \rightarrow a B D h$$

$$B \rightarrow c C$$

$$C \rightarrow b c \mid \epsilon$$

$$D \rightarrow E F$$

$$E \rightarrow g \mid \epsilon$$

$$F \rightarrow f \mid \epsilon$$

$$D \rightarrow E F$$

$$B \rightarrow c C$$

$$C \rightarrow b c$$

$$S \rightarrow a B D h$$

$$E \rightarrow g$$

$$F \rightarrow f$$

First:

$$\text{FIRST}(S) = \{a\}$$

$$\text{FIRST}(B) = \{c\}$$

$$\text{FIRST}(C) = \{b\}$$

$$\text{FIRST}(D) = \{g, \epsilon\}$$

$$\text{FIRST}(E) = \{g, \epsilon\}$$

$$\text{FIRST}(F) = \{f, \epsilon\}$$

Follow:

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(B) = \{g, \$ \}$$

$$\text{FOLLOW}(C) = \{g, \$ \}$$

$$\text{FOLLOW}(D) = \{g, \$ \}$$

$$\text{FOLLOW}(E) = \{g, \$ \}$$

$$\text{FOLLOW}(F) = \{g, \$ \}$$

⑦ Given Grammar:

$$S \rightarrow Sa \mid sb \mid ad.$$

⑧ Given:

$$E \rightarrow TE', E' \rightarrow +TE' \mid \epsilon$$

FIRST:

$$\text{FIRST}(E) = \{T\}$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

Answer:

$$\text{FIRST}(E') = \{+, \epsilon\}$$

⑨ Given:

$$S \rightarrow (S)S \mid \epsilon \quad \text{check string (w) is valid.}$$

⑩ Actions are performed in shift Reduction:

Shift-Reduce parsing.

1) Shift $\rightarrow$ shift, current symbol from I/P buffer on the stack

2) Reduce $\rightarrow$ pops out the RHS and push the LHS to the product

3) Error $\rightarrow$ If the stack cannot perform shift or reduce

4) Accept $\rightarrow$ Accept all production including ϵ .

⑪ Given:

$$S' \rightarrow S$$

$$S \rightarrow c c$$

$$c \rightarrow c c \mid d$$

$$\text{Item: } \{S' \rightarrow \cdot S\}$$

closure:

$$S' \rightarrow .S$$

$$S' \rightarrow .S$$

$$S \rightarrow .cC$$

$$S \rightarrow .cC \mid .d$$

$$\text{closure}(I_0, S') \equiv I_1$$

$$S' \rightarrow S.$$

$$\text{closure}(I_0, S) \equiv I_2$$

$$S \rightarrow c.C$$

$$\text{closure}(I_0, c) \equiv I_3$$

$$c \rightarrow c.C$$

$$c \rightarrow d.\checkmark$$

$$\text{closure}(I_2, S) \equiv I_4$$

$$S \rightarrow cC.\checkmark$$

$$\text{closure}(I_3, c) \equiv I_5$$

$$c \rightarrow cC.\checkmark$$

$$c \rightarrow d.\checkmark$$

$\therefore S' \rightarrow .S \therefore$ closure is accepted.

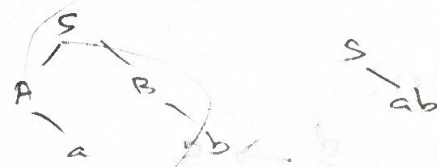
Q2 Given:

$$S \rightarrow AB, A \rightarrow a, B \rightarrow b.$$

Grammar:

$$S \rightarrow aSb \mid ab$$

String $aab b$.



FIRST:

$$\text{FIRST}(S) = \{A\}$$

$$\text{FIRST}(A) = \{a\}$$

$$\text{FIRST}(B) = \{b\}$$

parse tree:

$\therefore$ parse tree is not generated

string is not accepted.

13) Given:

$$S \rightarrow A$$

$$(A \rightarrow AB \mid Ad, A \rightarrow aB) \quad A \rightarrow dA' \text{ / } \cancel{A}, A' \rightarrow aBA' \mid \epsilon \quad A \rightarrow dA' \text{ / } \cancel{A} \text{ / } \cancel{B} \text{ / } \cancel{B}$$

$$B \rightarrow b$$

$$C \rightarrow g$$

$$aBA' \text{ / } \cancel{a} \text{ / } \cancel{B} \text{ / } \cancel{B}$$

FIRST:

$$\text{FIRST}(S) = \{A\}$$

$$\text{FIRST}(A) = \{d\}$$

$$\text{FIRST}(B) = \{b\}$$

$$\text{FIRST}(C) = \{g\}$$

$$\text{FIRST}(A') = \{a, \epsilon\}$$

FOLLOW:

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{\$ \}$$

$$\text{FOLLOW}(A') = \{\$ \}$$

$$\text{FOLLOW}(B) = \{a, \$ \}$$

$$\text{FOLLOW}(C) = \{a, \$ \}$$

14) Lex code to parse expressions with addition:

#include <stdio.h>

$a \rightarrow b + c$

int main() {

int a = 10, b = 20

int sum = a + b;

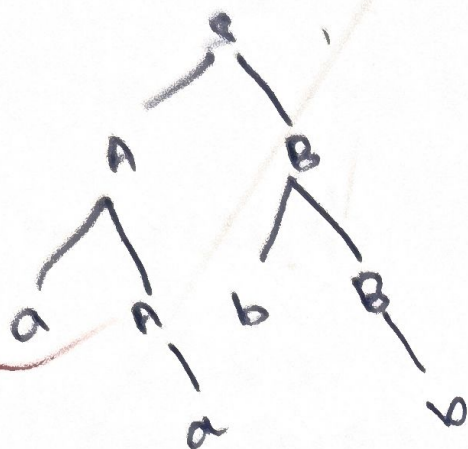
printf("%d", sum);

15) Given:

$$S \rightarrow AB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$



16) $S \rightarrow AB$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bB \mid c$$

FOLLOW & FIRST

FIRST

$$\text{FIRST}(S) = \{a, \epsilon\}$$

$$\text{FIRST}(A) = \{a, \epsilon\}$$

$$\text{FIRST}(B) = \{b, c\}$$

$$A \rightarrow aA \mid \epsilon$$

$$S \rightarrow AB \mid \epsilon$$

$$B \rightarrow bB \mid c$$

FOLLOW:

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{\$ \}$$

$$\text{FOLLOW}(B) = \{\$ \}$$

17) Rule of Left Recursion & Left factoring:

Left Recursion:

$$A \rightarrow \alpha A \mid \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

Left factoring:

$$A \rightarrow \alpha B \mid \beta$$

18) $Pd + Pd * Pd$

Grammar :

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow (E) / Pd$$

$$\begin{array}{c} TE' \\ \downarrow \\ BP \end{array}$$

$$\begin{array}{c} +TE' \\ \downarrow \\ BP \end{array}$$

$$\begin{array}{c} FT' \\ \downarrow \\ BP \end{array}$$

9) FIRST & FOLLOW :

$$FIRST(E) = \{ (, Pd \}$$

$$FIRST(E') = \{ +, \epsilon \}$$

$$FIRST(T) = \{ (, Pd \}$$

$$FIRST(T') = \{ *, \epsilon \}$$

$$FIRST(F) = \{ (, Pd \}$$

$$FOLLOW(E) = \{ \$ \}$$

$$FOLLOW(E') = \{ \$ \}$$

$$FOLLOW(T) = \{ +, \$ \}$$

$$FOLLOW(T') = \{ +, \$ \}$$

$$FOLLOW(F) = \{ *, +, \$ \}$$

$$\{ *, +, \$ \}$$

10)

Terminal Non Terminal	Pd	(	)	*	+	ε / \$
E	$E \rightarrow TE'$	$E \rightarrow TE'$				
E'					$E' \rightarrow +TE'$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	$T \rightarrow FT'$				
T'			$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow (E)$	$F \rightarrow (E)$				

iii)

Stack

\$

pd + pd * pd \$

-

\$ ~~pd~~

~~pd~~ + pd * pd \$

$E \rightarrow TE'$

\$ E' T

pd + pd * pd

$T \rightarrow T'F$

\$ E' T' F

~~pd~~ + pd * pd \$

$F \rightarrow pd$

\$ E' T ~~pd~~

+ pd * pd \$

$T' \rightarrow \epsilon$

\$ E'

* pd * pd \$

$E' \rightarrow +TE'$

\$ E' T ~~x~~

pd * pd \$

$E' \rightarrow \epsilon$

\$ T

pd * pd \$

FT'

\$ T' F

~~pd~~ * pd \$

$F \rightarrow pd$

\$ T ~~pd~~

* pd \$

$T' \rightarrow *FT'$

\$ T' F *

pd \$

$T' \rightarrow \epsilon$

\$ F

~~pd~~ \$

$F \rightarrow pd$
 $E \rightarrow pd$

\$ ~~pd~~

\$

Accepted

\$

\$

Accepted

b) Shift Reduction:

$id + id * id$

$id, (), *, +, E, \$$

LHS	Stack	RHS
\$	$id + id * id \$$	shift
$\$ id$	$+ id * id \$$	Reduce shift $E \rightarrow id$
$\$ id * E$	$+ id * id \$$	shift
$\$ E +$	$id * id \$$	Reduce $E \rightarrow id$
$\$ E + id$	$* id \$$	shift
$\$ E + E$	$* id$	shift
$\$ E$	$id \$$	Reduce $E \rightarrow E * E$
$\$ E *$	$id \$$	Accept
$\$ E * id$	$\$$	Accept
$\$ E * E$	$\$$	Accept

10

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$id + id * id$

LHS	Stack	RHS
\$	$id + id * id \$$	Shift
$\$ id$	$+ id * id \$$	Reduce or $E \rightarrow (E) id$
$\$ E$	$+ id * id \$$	Shift
$\$ E +$	$id * id \$$	Remove $E \rightarrow id$
$\$ E + E$	$* id \$$	Shift

\$ E + E	* Pd \$	$E \rightarrow E + E$
\$ E	* Pd \$	shift
\$ E *	Pd \$	Reduce $E \rightarrow Pd$
\$ E * E	* Pd \$	Reduce $E \rightarrow E * E$
\$ E	\$	Accepted
\$	\$	

② # include <stdio.h>

int main() {

int a = 10, b = 20;

int sum = a + b;

printf("%d", sum);

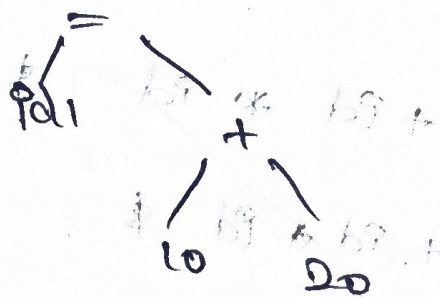
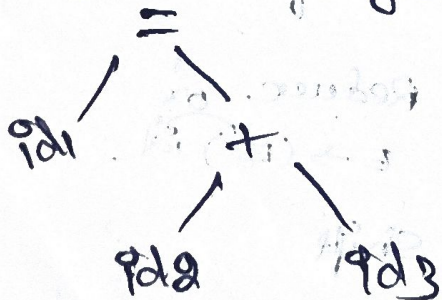
i) lexical analysis

sum = a + b

<sum> = <a> <+>

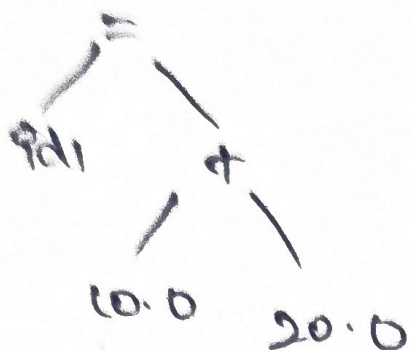
<Pd1> = <Pd2> <+> <Pd3>

ii) syntax analysis



iii) Semantic Analysis.

(Int to real)



iii) Intermediate code Generator

$t_1 = a + b$ (Int to real)

$id_1 = 10.0 + 20.0$

$id_1 = t_1$

iv) Code optimization

$t_1 = a + b$

$id_1 = t_1$

v) Code Generator

LDAF R₂, id₃

LDAF

~~ADDI R₁, R₁, 2~~

~~ADDF R₁, R₁, R₂~~

STF R₁